

Lab Work 5
Course Title: Information Security Laboratory/Project
Course Code: 0714 02 CSE 4108
Name: Sohana Rahman Tuly
Student ID: 230217

Implementation of ElGamal Digital Signature Scheme:

Source code:

```
def check_prime(n):
    if n < 2:
        return False
    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor += 1
    return True

def modular_power(base, power, mod):
    answer = 1
    for _ in range(power):
        answer = (answer * base) % mod
    return answer

def contains(items, target):
    for element in items:
        if element == target:
            return True
    return False

def check_primitive_root(g, p):
    if g < 2 or g >= p:
        return False
    generated = []
    current = 1
    for _ in range(1, p):
        current = (current * g) % p
        if contains(generated, current):
            return False
```

```

        generated.append(current)
    return True

def find_gcd(x, y):
    while y:
        x, y = y, x % y
    return x

def find_inverse(a, mod):
    for value in range(1, mod):
        if (a * value) % mod == 1:
            return value
    return -1

def main():
    q = int(input("Enter the prime number q : "))
    while True:
        if check_prime(q) and q >= 5:
            break
        q = int(input("q must be a prime number greater than 3,
enter q again : "))
    alpha = int(input("Enter the primitive root alpha : "))
    while not check_primitive_root(alpha, q):
        alpha = int(input(str(alpha) + " is not a primitive root
of " + str(q) + ", enter alpha again : "))
    xa = int(input("Enter the private key XA : "))
    while xa <= 1 or xa >= q - 1:
        xa = int(input("XA must satisfy 1 < XA < q-1, enter XA
again : "))
    ya = modular_power(alpha, xa, q)
    m = int(input("Enter the hash of the message m : "))
    while m < 0 or m > q - 1:
        m = int(input("m must satisfy 0 <= m <= q-1, enter m
again : "))
    k = int(input("Enter the random integer K : "))
    while k < 1 or k > q - 1 or find_gcd(k, q - 1) != 1:
        k = int(input("K must satisfy 1 <= K <= q-1 and gcd(K, q-
1) = 1, enter K again : "))
    s1 = modular_power(alpha, k, q)

```

```

k_inverse = find_inverse(k, q - 1)
s2 = (k_inverse * (m - xa * s1)) % (q - 1)
v1 = modular_power(alpha, m, q)
v2 = (modular_power(ya, s1, q) * modular_power(s1, s2, q)) %
q
print()
print(" Public key YA =", ya)
print(" Public key set {q, alpha, YA} = {" + str(q) + ", " +
str(alpha) + ", " + str(ya) + "}")
print(" K inverse mod (q-1) =", k_inverse)
print(" S1 =", s1)
print(" S2 = ", s2)
print("Signature Verification")
print(" V1 = ", v1)
print(" V2 = ", v2)
if v1 == v2:
    print("V1 = V2, the signature is valid")
else:
    print("V1 is not equal to V2, the signature is invalid")

if __name__ == "__main__":
    main()

```

Input and Output:



Problems Output Debug Console Terminal Ports

Python + - [] [X] ... [] [X]

```

PS D:\4_1\4.1 Term\Information Security\23\lab> & C:\Users\sohan\AppData\Local\Programs\Python\Python314\python.exe "d:/4_1/4.1 Term/Information Security/23/lab/ElGamalDigitalSignatureScheme.py"
Enter the prime number q : 19
Enter the primitive root alpha : 10
Enter the private key XA : 16
Enter the hash of the message m : 14
Enter the random integer K : 5

Public key YA = 4
Public key set {q, alpha, YA} = {19, 10, 4}
K inverse mod (q-1) = 11
S1 = 3
S2 = 4
Signature Verification
V1 = 16
V2 = 16
V1 = V2, the signature is valid
PS D:\4_1\4.1 Term\Information Security\23\lab>

```

Figure 1: ElGamal digital signature generation and verification.